

A PARALLEL VARIABLE MEMORY BFGS TRAINING ALGORITHM

Seán Mc Loone

*Intelligent Systems and Control Research Group, School of Electrical and Electronic
Engineering, The Queen's University of Belfast, Belfast BT9 5AH.
Tel.: 01232 274535, Email: s.mcloone@ee.qub.ac.uk*

Abstract: This paper considers the parallel implementation of a novel variable memory quasi-newton neural network training algorithm recently developed by the author. Unlike existing training methods this new technique is able to optimize performance in relation to available memory. Numerically it has equivalent properties to Full Memory BFGS optimization (FM) when there are no restrictions on memory and to FM with periodic reset when memory is limited. Parallel implementations of both the Full and Variable Memory BFGS algorithms are outlined and performance results presented for a PVM target architecture. *Copyright © 2000 IFAC*

Keywords: Neural networks, parallel algorithms, training, second-order.

1 INTRODUCTION

Feedforward neural network training is usually posed as an unconstrained optimization problem where the objective is to minimise a mean-squared-error cost function

$$E(\mathbf{w}) = \frac{1}{N_v} \sum_{i=1}^{N_v} (y(\mathbf{w}, \mathbf{u}_i) - d_i)^2 \quad (1)$$

defined on a training set

$$\tau = \{\mathbf{u}_i, d_i\}, \quad i \in 1, N_v \quad (2)$$

with respect to the network weights $\mathbf{w}$. Here $\mathbf{w}$ is an N_w x 1 vector, N_v is the size of the training set and $\mathbf{u}_i, d_i$ and $y(\mathbf{w}, \mathbf{u}_i)$ are the input vector, desired network output and actual network output for the i^{th} training vector respectively.

Training is achieved by updating the weights using a learning rule of the form

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \eta_k \mathbf{d}_k \quad (3)$$

where $\mathbf{d}_k$ is a descent search direction and η_k is the step size or "learning rate" at the k^{th} iteration.

The choice of descent direction, $\mathbf{d}_k$, dictates the rate of convergence achievable with a given technique and is typically a trade-off between performance and

computational/memory requirements. In the Batch Back Propagation (BBP) algorithm, the original method developed for training MLPs (Rumelhart *et al.* (1986)), $\mathbf{d}_k$ is simply the negative of the cost function gradient at $\mathbf{w}_k$, that is:

$$\mathbf{d}_k = -\mathbf{g}_k \quad (4)$$

where

$$\mathbf{g}_k = \left[\frac{\partial E(\mathbf{w})}{\partial w_1} \quad \frac{\partial E(\mathbf{w})}{\partial w_2} \quad \dots \quad \frac{\partial E(\mathbf{w})}{\partial w_{N_w}} \right]^T \quad (5)$$

while in the Full Memory BFGS (FM) algorithm, a powerful second order technique, the search direction is given by

$$\mathbf{d}_k = -\mathbf{M}_k \mathbf{g}_k \quad (6)$$

where $\mathbf{M}_k$ is a symmetric positive definite approximation to the inverse Hessian matrix of the cost function that is updated iteratively as follows.

$$\mathbf{s}_k = \mathbf{w}_k - \mathbf{w}_{k-1}, \quad \mathbf{t}_k = \mathbf{g}_k - \mathbf{g}_{k-1} \quad (7)$$

$$\mathbf{M}_k = \mathbf{M}_{k-1} + \left[1 + \frac{\mathbf{t}_k^T \mathbf{M}_{k-1} \mathbf{t}_k}{\mathbf{s}_k^T \mathbf{t}_k} \right] \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{t}_k} - \frac{\mathbf{s}_k \mathbf{t}_k^T \mathbf{M}_{k-1} + \mathbf{M}_{k-1} \mathbf{t}_k \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{t}_k}, \quad \mathbf{M}_0 = \mathbf{I} \quad (8)$$

The FM algorithm is typically two orders of magnitude faster than BBP but this is achieved at the expense of $O(N_w^2)$ memory requirements and computational complexity. Consequently FM cannot be used for large dimensional problems. Conjugate gradient or Memory-less BFGS algorithms, which only require $O(N_w)$ storage, can be used instead, but these are only one order of magnitude faster than BBP (McLoone and Irwin (1997)). This leads to very poor utilisation of resources, especially when the available memory is only marginally less than that required for FM. Clearly much better utilisation of resources could be achieved if the performance of a training algorithm varied as a function of available memory. The variable memory (VM) BFGS algorithm recently developed by McLoone and Irwin (1999) has this property. Instead of storing and updating the inverse Hessian approximation ($\mathbf{M}_k$) at each iteration VM BFGS stores only those vectors and scalars needed to compute the product $\mathbf{M}_k \mathbf{g}_k$. This yields the following set of equations for the search direction calculation.

$$\mathbf{u}_k = \mathbf{t}_k - \sum_{j=1}^{k-1} \mathbf{f}_j(\mathbf{t}_k) \quad (9)$$

$$n_k = \mathbf{s}_k^T \mathbf{t}_k, \quad v_k = \mathbf{t}_k^T \mathbf{u}_k \quad (10)$$

and

$$\mathbf{d}_k = -\mathbf{g}_k + \sum_{j=1}^k \mathbf{f}_j(\mathbf{g}_k) \quad (11)$$

where

$$\mathbf{f}_j(\mathbf{r}) = \left[\frac{\mathbf{u}_j^T \mathbf{r}}{n_j} - \left(1 + \frac{v_j}{n_j} \right) \frac{\mathbf{s}_j^T \mathbf{r}}{n_j} \right] \mathbf{s}_j + \left[\frac{\mathbf{s}_j^T \mathbf{r}}{n_j} \right] \mathbf{u}_j \quad (12)$$

In order to compute $\mathbf{d}_k$ at the k^{th} iteration, the scalars v_j , n_j and vectors $\mathbf{u}_j$ and $\mathbf{s}_j$ have to be stored for all the previous iterations. Thus, the memory requirements and computational complexity of VM increase linearly with iteration number and are therefore $O(kN_w)$. Consequently when available memory is limited to $O(L_{\text{mem}}N_w)$, where $L_{\text{mem}} \ll N_w$, a situation which precludes FM, the VM alternative can be used if it is reset every time the memory limit is reached. VM is numerically equivalent to FM with reset and its performance varies between that of the Full and Memory-less BFGS algorithms as a function of the resetting frequency.

Neural networks are naturally parallel structures, therefore it is not surprising that parallel implementations of these architectures and their training algorithms have been the subject of a great deal of research over the past decade with implementations reported across the full spectrum of concurrent architectures, from Single Instruction Multiple Data (SIMD) machines such as the

Connection Machine (CM) and MasPar to Multiple Instruction Multiple Data (MIMD) machines which include Transputer networks and workstation clusters. Examples include a parallel back-propagation algorithm by Besch *et al.* (1994), a Connection Machine based partial conjugate gradient algorithm by Kramer (1988), a transputer based parallel Memory-less BFGS algorithm by Lightbody and Irwin (1992) and PVM based implementation of the Full Memory BFGS algorithm by McLoone and Irwin (1997).

The intention in this paper is to extend the work of McLoone and Irwin (1997) on Full Memory BFGS to cover the new Variable Memory training algorithm. In that contribution a parallel implementation of FM incorporating matrix partitioning of the search direction calculations and training data partitioning of the cost function and gradient calculations was identified as being the most appropriate for the problem dimensions typically encountered in engineering system identification and function approximation applications. Since VM only differs from FM in the formulation of the search direction calculation this paper will only consider parallel implementation of this segment of the algorithm. Accordingly the remainder of the paper is structured as follows.

In section 2 efficient sequential and parallel implementations of the FM BFGS update equations (6 - 8) are presented and a timing analysis is used to highlight the significant contribution these equations make to the total iteration time. Section 3 compares the performance of the sequential VM and FM BFGS updates with the aid of benchmark training problems from the Proben benchmark suite (Prechelt (1994)). Two different parallel implementations of the VM algorithm are also presented in section 3. The first is based on placing the data associated with each iteration on a different processor while the second is based on partitioning individual vectors over all the processors. Finally in section 4 numerical results are presented comparing the performance of the various algorithms considered for a PVM target architecture consisting of a cluster of 4 Sun workstations.

2 FULL MEMORY BFGS

2.1 Sequential FM BFGS update algorithm

To obtain an efficient sequential implementation the BFGS update calculations (6 - 8) are generally expressed in the form

$$\mathbf{s}_k = \mathbf{w}_k - \mathbf{w}_{k-1}, \quad \mathbf{t}_k = \mathbf{g}_k - \mathbf{g}_{k-1} \quad (13)$$

$$n_k = \mathbf{s}_k^T \mathbf{t}_k, \quad \mathbf{u}_k = \mathbf{M}_{k-1} \mathbf{t}_k, \quad (14)$$

$$v_k = \mathbf{t}_k^T \mathbf{u}_k, \quad \phi_k = \frac{n_k^2}{n_k + v_k} \quad (15)$$

$$\mathbf{M}_k = \mathbf{M}_{k-1} + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\phi_k} - \frac{\mathbf{u}_k \mathbf{s}_k^T + \mathbf{s}_k \mathbf{u}_k^T}{n_k} \quad (16)$$

$$\mathbf{d}_k = -\mathbf{M}_k \mathbf{g}_k \quad (17)$$

where v_k , n_k and ϕ_k are scalars and $\mathbf{u}_k$ is a vector. This formulation allows exploitation of the symmetry in $\mathbf{M}$ so that only the lower diagonal elements of $\mathbf{M}$ need to be computed and stored, that is:

$$[m_{ij}]_k = [m_{ij}]_{k-1} + \frac{s_i s_j}{\phi_k} - \frac{u_i s_j + s_i u_j}{n_k} \quad (18)$$

for $i=1$ to N_w , $j=1$ to i . The memory requirements and complexity of the symmetric FM BFGS update are thus

$$\mu_{\text{SFM}} \equiv \frac{N_w(N_w + 7)}{2} \quad (19)$$

$$\chi_{\text{SFM}} \equiv 2N_w(4N_w + 5) \quad (20)$$

Here μ is defined as the number of storage locations and χ is defined as the number of arithmetical operations (multiplications, divisions, additions or subtractions) required by the equations. While there are a number of $O(N_w)$ operations in the update the $O(N_w^2)$ operations dominate for large N_w .

2.2 Timing analysis

The principle components of the Full Memory BFGS training algorithm are the batch gradient calculation which has $O(N_v N_w)$ complexity, the step size calculation which has $O(N_c N_v N_w)$ complexity and the BFGS search direction calculation outlined above. (N_v and N_w are as previously defined and N_c is the average number of cost function evaluations needed by the step size estimation routine). Figure 1 shows a graph of the average percentage contribution of each of these components to the overall algorithm iteration time plotted as a function of the ratio N_w/N_v . The boundary curves have been estimated from timing results obtained for a range of MLP network and training set sizes trained using a Matlab 5[®] implementation of the algorithm. In all cases the remaining computations amounted to less than 0.1% of the total iteration time and have omitted. From the graph it can be seen that the line search computation is the most computationally intensive element of the training algorithm when the ratio N_w/N_v is small, but that the influence of the BFGS update increases with N_w/N_v and dominates for $N_w/N_v > 0.5$. Note that for $N_w/N_v = 0.2$ the BFGS update and line search amount to 30% and 60% of the computation respectively but that the situation is reversed for $N_w/N_v = 0.35$.

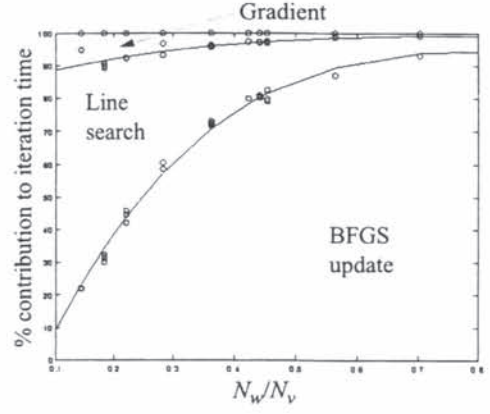


Fig. 1. Cumulative sum of the % contributions of the BFGS update, line search and gradient calculations to the total iteration time plotted as a function of the ratio of the number of weights N_w to the number of training vectors N_v .

2.3 Parallel implementation

Parallel implementation of the BFGS search direction equations can be achieved by partitioning the $\mathbf{M}$ matrix by rows so that N_w/p rows are placed on each of p slave processes. Defining the row partition of a matrix $\mathbf{A}$ and vector $\mathbf{x}$ resident on the r^{th} processor as

$$\begin{aligned} \Delta_r \mathbf{A} &= [a_{ij}], & \Delta_r \mathbf{x} &= [x_i] \\ i &= \left[1 + \frac{N_w}{p}(r-1) \right] \text{ to } \left[\frac{N_w}{p}r \right] \\ j &= 1 \text{ to } N_w \end{aligned} \quad (21)$$

the parallel FM BFGS update algorithm can be summarised as follows:

At the k^{th} iteration, after the gradient has been determined, $\Delta_r \mathbf{M}_{k-1}$, $\mathbf{w}_k$, $\mathbf{w}_{k-1}$ and $\mathbf{g}_{k-1}$ are present on the r^{th} slave processor and $\mathbf{g}_k$ is present on the master processor.

Step 1: The slave processors receive $\mathbf{g}_k$ from the master processor and compute $\mathbf{s}_k$ and $\mathbf{t}_k$.

Step 2: Equations

$$\begin{aligned} \Delta_r \mathbf{u}_k &= \Delta_r \mathbf{M}_k \mathbf{t}_k, \\ \Delta_r v &= \Delta_r \mathbf{t}_k^T \Delta_r \mathbf{u}_k \end{aligned} \quad (22)$$

are evaluated in parallel and the results transmitted to the master processor where they are used to obtain v_k and $\mathbf{u}_k$, that is:

$$\mathbf{u}_k = \left[\Delta_1 \mathbf{u}_k^T \dots \Delta_p \mathbf{u}_k^T \right]^T, \quad v_k = \sum_{r=1}^p \Delta_r v_k \quad (23)$$

Step 3: The slave processors receive v_k and $\mathbf{u}_k$ from the

master processor and compute the scalars n_k and ϕ_k .

$$n_k = \mathbf{s}_k^T \mathbf{t}_k, \quad \phi_k = \frac{n_k^2}{n_k + v_k} \quad (24)$$

Step 4: Each slave process then updates its portion of the $\mathbf{M}$ matrix ($\Delta_r \mathbf{M}_k$) using equation (18).

Step 5: Finally the slave processes compute $\Delta_r \mathbf{d}_k$, $r=1$ to p in parallel and transmit the results to the master processor where they are combined to give $\mathbf{d}_k$, that is:

$$\Delta_r \mathbf{d}_k = \Delta_r \mathbf{M}_k \mathbf{g}_k, \quad (25)$$

$$\mathbf{d}_k = \begin{bmatrix} \Delta_1 \mathbf{d}_k^T & \dots & \Delta_p \mathbf{d}_k^T \end{bmatrix}^T \quad (26)$$

The memory and computational requirements of this implementation of the FM BFGS update are given by

$$\mu_{\text{PFM}} \equiv N_w \left(\frac{N_w}{p} + 3 \right) \quad (27)$$

$$\chi_{\text{PFM}} \equiv \frac{2N_w}{p} (6N_w + 2p + 1) \quad (28)$$

but this is at the expense of communication overheads (τ) which are approximately given by

$$\tau_{\text{PFM}} \equiv 2[N_w(p+1) + p]\tau_c + 4p\tau_o \quad (29)$$

where τ_c and τ_o are the mean transfer time per floating point number and mean communication initiation time respectively.

The symmetry in $\mathbf{M}$ cannot be exploited when employing standard row partitioning as given in equation (21). However if the partitioning of $\mathbf{M}$ is defined as

$$\Delta_r \mathbf{M}^* = [m_{ij}] \quad (30)$$

$$\left\{ i = \left[1 + \frac{N_w}{p}(r-1) \right] \text{ to } \left[\frac{N_w}{p}r \right], j = 1 \text{ to } i \right\} \quad (31)$$

$$\left\{ i = \left[\frac{N_w}{p}r + 1 \right] \text{ to } N_w, j = \left[1 + \frac{N_w}{p}(r-1) \right] \text{ to } \left[\frac{N_w}{p}r \right] \right\}$$

symmetry can be exploited leading to reduced memory requirement and complexity. The difference between the partitioning strategies is illustrated graphically in Figure 2. Partitions $\Delta_r \mathbf{M}^*$ have fewer elements than $\Delta_r \mathbf{M}$, but contain the same information because of symmetry. The reduced memory usage and complexity arising from this modification are

$$\mu_{\text{PSFM}} \equiv \mu_{\text{PFM}} - \frac{N_w}{2p} \left(\frac{N_w}{p} - 1 \right) \quad (32)$$

$$\chi_{\text{PSFM}} \equiv \chi_{\text{PFM}} - \frac{4N_w}{p} \left(\frac{N_w}{p} - 1 \right) \quad (33)$$

Note that the reductions are only significant when the partitions N_w/p are large.

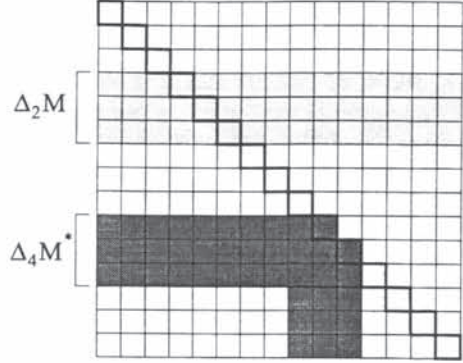


Fig. 2. Graphical representation of matrix partitioning strategies $\Delta_r \mathbf{M}$ and $\Delta_r \mathbf{M}^*$ applied to a 15 x 15 matrix divided over 5 processors.

3 VARIABLE MEMORY BFGS UPDATE

3.1 Performance of the sequential VM algorithm

The memory requirements and complexity of the VM BFGS update equations (9) to (12) at the k^{th} iteration are given by

$$\mu_{\text{VM}} \equiv 2k(N_w + 1) + N_w \quad (34)$$

$$\chi_{\text{VM}} \equiv 2k(7N_w + 6) + N_w \quad (35)$$

Comparing these expressions with the corresponding expressions for the Full Memory BFGS update (equations (19) and (20)) it can be shown that:

$$\mu_{\text{VM}} < \mu_{\text{SFM}} \text{ if } k < \frac{N_w}{4} \quad (36)$$

$$\chi_{\text{VM}} < \chi_{\text{SFM}} \text{ if } k < \frac{4N_w}{7} \quad (37)$$

$$\sum_{k=1}^{N_k} \chi_{\text{VM}} < \sum_{k=1}^{N_k} \chi_{\text{SFM}} \text{ if } N_k < \frac{8N_w}{7} \quad (38)$$

Equations (36) and (37) define the conditions under which VM uses less memory and has shorter iteration times than FM, while equation (38) indicates the number of iterations N_k that can be computed before the total training time for VM exceeds that of FM. In practice when using FM BFGS neural network training algorithms termination occurs in significantly less than N_w iterations. Therefore by equation (38) VM BFGS will always be computationally superior to FM

BFGS. If the VM algorithm is reset every L iterations where $L \ll N_w$ mean iteration times will be much quicker than those of FM, but convergence will generally require more iterations. In many cases the convergence versus iteration time trade off is such that VM with reset will outperform VM without reset. To illustrate this point a 10 neuron MLP was trained for, two test problems from the Proben benchmark suite (Prechelt (1994)) (*build1a* and *flare1a*) using the FM algorithm and various flavours of VM. Training was halted in each case when an error goal of 0.2% was achieved or the number of iterations exceeded 1000. The number of iterations and total training time for each algorithm are recorded in table 1 below. As expected VM is superior to FM in both problems. This is most evident in *flare1a* where the BFGS update represents a significant portion of the computation (i.e. $N_w/N_v > 0.5$). Note also that VM with a resetting factor of 70 outperforms VM in problem *build1a*.

Table 1: No. of iterations and training times for FM, VM and VM with periodic reset

Alg.	build1a $N_w=183, N_v=2100$		flare1a $N_w=283, N_v=530$	
	No. of iterations	Time (secs)	No. of iterations	Time (secs)
FM	148	144.7	146	148.8
VM	148	122.9	146	48.0
VM5	1000+	628.1	1000+	227.6
VM10	1000+	662.9	1000+	238.6
VM20	479	350.2	297	77.1
VM30	237	181.2	206	55.0
VM40	197	155.2	187	51.4
VM50	168	134.2	182	52.0
VM60	153	121.2	178	51.2
VM70	149	118.8	172	50.3

3.2 Parallel VM BFGS update implementations

Since the computational complexity and memory requirements of the Variable Memory BFGS update are $O(kN_w)$, parallel implementation is possible either by partitioning the data in terms of k or in terms of N_w .

Partitioning in terms of k . Here, the data associated with each iteration is stored on a different processor so that at the k^{th} iteration $\mathbf{u}_j, \mathbf{s}_j, n_j$ and $v_j, j=1$ to $k-1$, are distributed over the p parallel processors with the j^{th} components resident on the ϕ_j^{th} processor where ϕ_j is defined as

$$\phi_j = (j-1) \bmod p + 1 \quad (39)$$

Using this approach the $\mathbf{f}_j(\cdot)$ expressions in equations

(9) and (11) can be determined in parallel. The algorithm proceeds as follows.

After completing the gradient calculation at the k^{th} iteration $\mathbf{w}_k, \mathbf{w}_{k-1}$ and $\mathbf{g}_{k-1}$ are present on the slave processors and $\mathbf{g}_k$ is present on the master processor.

Step 1: The slave processors receive $\mathbf{g}_k$ from the master processor and compute $\mathbf{t}_k$.

Step 2: Each slave process evaluates the partial sum $\sum \mathbf{f}_j(\mathbf{t}_k)$ for the data $(\mathbf{u}_j, \mathbf{s}_j, n_j$ and $v_j)$ currently stored on it and transmits the result to the ϕ_k^{th} processor.

Step 3: The ϕ_k^{th} processor collects the partial sums from the parallel processes and uses them to compute $\mathbf{u}_k$. Vector $\mathbf{s}_k$ and scalars n_k and v_k are also computed and stored along with the $\mathbf{u}_k$ for future use.

Step 4: Partial sums $\sum \mathbf{f}_j(\mathbf{g}_k)$ are determined in parallel and transmitted to the ϕ_k^{th} processor where the search direction $\mathbf{d}_k$ is computed using eqt. (11).

The memory requirements, computational complexity and inter processor communication overheads are approximately given by

$$\mu_{\text{PVM}k} \cong \frac{2k}{p}(N_w + 1) + N_w \quad (40)$$

$$\chi_{\text{PVM}k} \cong \frac{N_w}{p}(14k + 6p + 2p^2) + \frac{12k}{p} \quad (41)$$

$$\tau_{\text{PFM}k} \cong 3pN_w\tau_c + 3p\tau_o \quad (42)$$

This partitioning strategy results in a factor p increase in storage capacity, hence if k_s full BFGS updates can be accommodated when using the sequential algorithm, the partitioned algorithm can compute pk_s updates. However, computationally the strategy is very poor since parallelisation is incomplete and processing power is under utilised.

Partitioning in terms of N_w : In this approach individual vectors are partitioned over all the processors leading to the following parallel algorithm:

At the k^{th} iteration, $\Delta_r \mathbf{u}_j, \Delta_r \mathbf{s}_j, n_j$ and $v_j, j=1$ to $k-1$ are distributed over the p parallel processors. The gradient $\mathbf{g}_k$ is present on the master processor and $\mathbf{w}_k, \mathbf{w}_{k-1}$ and $\mathbf{g}_{k-1}$ are available on each slave process.

Step 1: Each slave processor receives its segment of $\mathbf{g}_k$ ($\Delta_r \mathbf{g}_k \rightarrow$ processor r) and computes $\Delta_r \mathbf{t}_k$ and $\Delta_r \mathbf{s}_k$.

Step 2: Partial sums $\Delta_r \mathbf{u}_j^T \Delta_r \mathbf{t}_k$ and $\Delta_r \mathbf{s}_j^T \Delta_r \mathbf{t}_k, j=1$ to $k-1$ are computed by the slave processes and transmitted to the master processor where they are summed to give $\mathbf{u}_j^T \mathbf{t}_k$ and $\mathbf{s}_j^T \mathbf{t}_k, j=1$ to $k-1$.

Step 3: The slave processes receive $\mathbf{u}_j^T \mathbf{t}_k$ and $\mathbf{s}_j^T \mathbf{t}_k$, $j=1$ to $k-1$ from the master processor and hence compute $\Delta_r \mathbf{u}_k$, $\Delta_r \mathbf{v}_k$ and $\Delta_r \mathbf{s}_k$ are stored for future use.

Step 4: The slave processes compute the partial sums $\Delta_r \mathbf{n}_k$ and $\Delta_r \mathbf{v}_k$ and transmit them to the master processor where they are combined to give $\mathbf{n}_k$ and $\mathbf{v}_k$. These quantities are then returned to the slave processes where they are stored for future use.

Step 5: Following the procedure outlined in step 2 the slave processes determine $\mathbf{u}_j^T \mathbf{g}_k$ and $\mathbf{s}_j^T \mathbf{g}_k$, $j=1$ to k and hence compute $\Delta_r \mathbf{d}_k$. Finally the $\Delta_r \mathbf{d}_k$ partitions are transmitted to the master process where they are combined to give $\mathbf{d}_k$.

The memory, complexity and communication costs associated with this implementation are

$$\mu_{\text{PVM}N_w} \equiv 2k \left(\frac{N_w}{p} + 2 \right) + N_w \quad (43)$$

$$\chi_{\text{PVM}N_w} \equiv \frac{N_w}{p} (14k + 1) + 4k(p + 3) \quad (44)$$

$$\tau_{\text{PVM}N_w} \equiv (2N_w + 8kp)\tau_c + 8p\tau_o \quad (45)$$

Comparing the expressions for both implementations it can be seen that provided the ratio N_w/p is large the second algorithm is superior. The trade-off is a slight reduction in memory capacity because the scalars n_k and v_k are needed on all processors.

4 SIMULATION RESULTS

To assess the performance of the parallel FM and VM BFGS updates discussed in the paper, speed-ups relative to the sequential FM BFGS algorithm have been determined for PVM implementations on a cluster of 4 Sun workstations. The results, which are based on the benchmark problems and training conditions described in section 3, are given in table 2.

Table 2: Parallel FM and VM BFGS speeds-ups relative to the sequential FM BFGS update.

Training Algorithm	Speed Up Relative to FM BFGS			
	<i>build1a</i>		<i>flare1a</i>	
	$p=2$	$p=4$	$p=2$	$p=4$
PFM	1.14	1.61	1.24	2.04
PSFM	1.33	1.70	1.46	2.18
VM	1.41	1.41	3.20	3.20
PVM k	1.68	1.56	4.42	4.68
PVM N_w	2.15	2.55	5.17	6.61

The speed-ups observed are consistent with predictions. The parallel FM BFGS algorithm which exploits symmetry (PSFM) out performing the version that doesn't (PFM) while the vector based partitioning strategy (PVM N_w) gives the best results for the VM algorithm. While the speeds-ups obtained are significant they fall well short of the theoretical maximum. This is mainly due to the influence of the communication overheads which are amplified by the relatively slow data transmission rates of the PVM architecture employed.

5 CONCLUSIONS

This paper has proposed parallel implementation strategies for the search direction calculation in the FM and VM BFGS training algorithms. In addition to facilitating larger problems sizes the proposed algorithms provide significant speed-ups relative to their sequential counterparts.

6 REFERENCES

- Besch M. & Pohl H.W. (1995), "Flexible data parallel training of neural networks using MIMD-computers", In: *Third Workshop on Parallel and Distributed Processing*, Sanremo, Italy.
- Kramer A.H. & Sangiovanni-Vincentelli (1988), "Efficient parallel learning algorithms for neural networks", *Advances in Neural Information Processing Systems*, Vol.1, Pages 75-89.
- Lightbody G. & Irwin G.W. (1992). A parallel Algorithm for Training Neural Network Based Nonlinear Models. In: *Proc. 2nd IFAC Workshop on Algorithms and Architectures for Real-time Control*, pp. 99-104.
- McLoone S.F. & Irwin G.W. (1999), 'A Variable memory Quasi-Newton Training Algorithm', *Neural Processing Letters*, Vol.9(1), pp. 77-89.
- McLoone S.F. & Irwin G.W. (1997), "Fast parallel off-line training of multilayer perceptrons", *IEEE Trans. Neural Networks*, Vol.8, No.3, pp. 646-653.
- Prechelt L. (1994), "Proben 1 - A set of neural network benchmark problems and benchmarking rules", Technical Report 21/94, Fakultät für Informatik, Universität Karlsruhe, 76128 Karlsruhe, Germany, Sept 1994. Available by anonymous FTP at ftp.ira.uka.de in /pub/neuron/proben1.tar.gz.
- Rumelhart D.E, G. Hinton & R. Williams (1986). "Learning internal representations by error propagation". In: *Parallel Distributed Processing* (D.E Rumelhart, J.L. McClelland, (Ed.)), Vol.1, pp. 318-364. MIT Press.